

RadicalRoot

From Algebraic Root Objects to Certified Radical Towers

A constructive Galois algorithm, compact structural methods,
and a Python implementation with a Wolfram Language adapter

Prepared for Vladimir Reshetnikov
Research/reference implementation, version 1.0.0

8 September 2026

Abstract

We develop and implement a systematic conversion of numerical algebraic numbers to arithmetic radicals. The input is an exact selected polynomial root, not a floating-point approximation and not a parameter-dependent root. A compact front end recognizes translated binomials, Dickson polynomials, monomial substitutions, cyclotomic polynomials, and generalized reciprocal substitutions. These methods solve both examples in the originating Mathematica question and a degree-eight example from the supplied experimental notebook.

The general back end is constructive rather than a placeholder for a Galois-group oracle. It builds a splitting field by exact quotient-field arithmetic, computes automorphisms from primitive-generator images, passes to a cyclotomic base, and converts prime-index normal steps into radicals by rational-matrix eigenspace computations. A separate verifier checks the resulting arithmetic program, including principal-power branches and the selected conjugate. With artificial resource limits removed, the mathematical algorithm is finite and complete over $\mathbb{Q}$, under the stated exact-arithmetic primitives. This is not a claim of practical tractability for every solvable input.

The delivered Python implementation was executed and tested. The Wolfram Language adapter calls this implementation and adds a final `RootReduce` equality check, but could not be runtime-tested in the delivery environment. Default limits distinguish `Unknown` from a proved `NotSolvable`. The article explains the notebook's useful ideas and limitations, proves the constructive steps, specifies the certificate formats, and reports both successful tests and expensive runs that did not finish within their time limits.

Scope. Radicals may be complex and may have arbitrary positive integer orders. No claim is made about shortest expressions, minimum radical depth, real-radical-only expressions, symbolic parameters, or production-grade performance.

Contents

1	The problem and the computational contract	4
1.1	What is being converted?	4
1.2	Use the minimal polynomial, not the entire defining polynomial	4
1.3	Four outcomes, not a boolean heuristic	5
2	What the supplied notebook contributes	5
2.1	Inspection and provenance	5
2.2	Extension discovery as a resolvent construction	5
2.3	Specific limitations visible in the input cells	6
3	Compact structural constructions	6
3.1	Translation, binomials, and monomial substitution	6
3.2	Dickson polynomials and coherent radical branches	7
3.3	Generalized reciprocal reduction	7
3.4	Cyclotomic and low-degree cases	7
3.5	Structural certificates and a dependency edge case	8
4	The original examples, and a notebook octic	8
4.1	The sextic reduces to a cubic and a quadratic	8
4.2	The quintic is a Dickson equation	9
4.3	A degree-eight example from the notebook	9
5	Exact splitting fields without numerical recognition	10
5.1	Representation and trusted primitives	10
5.2	Adjoining a root in a quotient field	10
5.3	Building the splitting field	11
6	A cyclotomic base that preserves the solvability decision	11
7	Computing the Galois action explicitly	12
7.1	Images of a tracked primitive generator	12
7.2	Small-degree and modular shortcuts	12
7.3	A prime-index normal series	12
8	Kummer generators by rational linear algebra	13
8.1	The linear-algebra lemma	13
8.2	Why the matrices have rational, not algebraic, entries	13
8.3	Converting the power into the preceding radical field	14

9	Exact embeddings and principal-power branches	14
9.1	Abstract fields may change their complex embedding	14
9.2	Rational rectangles and exact root matching	14
9.3	Complex conjugation and sign decisions	15
9.4	Fixing the cyclotomic generator	15
9.5	A branch condition expressible by exact signs	15
10	The complete algorithm and its correctness	16
10.1	Pseudocode	16
11	Certificates and independent checks	17
11.1	The radical program	17
11.2	Positive general certificates	17
11.3	Negative certificates	18
12	Package interfaces and reproducible use	18
12.1	Installation and Python API	18
12.2	Command-line interface and saved certificates	19
12.3	Wolfram Language adapter	19
13	Executed validation and observed limits	20
13.1	What was actually run	20
13.2	Expensive runs that must not be counted as successes	21
13.3	Limits of the evidence	21
14	Performance, expression quality, and further development	21
14.1	Where the reference implementation spends time	21
14.2	Improvements that preserve the mathematical contract	21
14.3	What remains outside the contract	22
15	Conclusion	22
A	File map and reproduction notes	22
B	The distinction between an expression and its proof	23

1 The problem and the computational contract

1.1 What is being converted?

Let $\alpha \in \overline{\mathbb{Q}} \subset \mathbb{C}$ be specified exactly. In Wolfram Language, a typical input is `Root[f &, k]`; in Python it is a polynomial together with a zero-based `CRootOf` index. The task is to construct a finite expression for the *specified embedding* of α , using rational numbers, arithmetic operations, and rational powers, whenever such an expression exists.

This is different from finding a polynomial satisfied by a plausible expression, different from approximating the number, and different from finding *some* conjugate. For example, both $\sqrt{2}$ and $-\sqrt{2}$ satisfy $X^2 - 2$, but they are not interchangeable in a selected-root problem.

Wolfram's documentation describes guaranteed cubic and quartic conversion and examples of higher-degree conversion; it does not make `ToRadicals` a complete constructive Galois solver [3]. We therefore retain it as an optional fast path, not as a decision procedure for radical solvability. An unchanged `Root` is not a proof that radicals are impossible.

Definition 1.1 (Allowed output). An arithmetic radical expression is obtained from $\mathbb{Q}$ by addition, multiplication, division by nonzero values, and $z \mapsto z^{a/b}$ with $a \in \mathbb{Z}$, $b \in \mathbb{Z}_{>0}$. For nonzero z , powers have the principal convention

$$z^{a/b} = \exp((a/b)(\log |z| + i \operatorname{Arg} z)), \quad -\pi < \operatorname{Arg} z \leq \pi.$$

The exponential and logarithm in this definition specify semantics; they are not output operations. The constant i is $(-1)^{1/2}$.

A sequential program is also an expression: intermediate values are defined once and subsequently referenced. Substitution of earlier definitions produces an ordinary nested radical expression. This representation avoids repeatedly copying large radicands. Neither `Root`, `CRootOf`, `AlgebraicNumber`, trigonometric functions, nor special inverse functions are allowed in the output expression tree. Such objects may occur in the *certificate*, where they describe exact algebraic data used to verify the expression.

1.2 Use the minimal polynomial, not the entire defining polynomial

Let $m_\alpha \in \mathbb{Q}[X]$ be the monic irreducible polynomial of α . A reducible input polynomial may have both solvable and nonsolvable factors. For instance,

$$(X - 2)(X^5 - X - 1)$$

has a rational root even though the quintic factor is nonsolvable. The Wolfram adapter computes `MinimalPolynomial`; the Python selected-root interface obtains the irreducible factor and local index from the exact `CRootOf`. Multiplicities in the original polynomial do not create new algebraic numbers.

Theorem 1.2 (The relevant solvability criterion). *For an algebraic number α , the following are equivalent: α belongs to a radical extension of $\mathbb{Q}$; every conjugate of α is expressible by radicals; and the Galois group of the splitting field of m_α over $\mathbb{Q}$ is solvable.*

The classical Galois solvability theorem provides the equivalence between radical solvability of an irreducible polynomial and solvability of its Galois group [2]. To see why one conjugate is enough,

extend each embedding of $\mathbb{Q}(\alpha)$ to a normal closure of a radical tower after adjoining the required roots of unity. Its conjugate towers contain every conjugate of α ; their compositum has solvable Galois group. Conversely, our construction below explicitly expresses every root when the splitting group is solvable.

1.3 Four outcomes, not a boolean heuristic

Result	Meaning
Success	A radical program has been constructed, with a positive certificate.
NotSolvable	An exact nonsolvable Galois group or a certified group-theoretic obstruction has been obtained.
Unknown	A resource limit was reached, or the explicitly requested structural-only method did not apply.
Error	Invalid input, an incompatible dependency, or a failed implementation/backend invariant. This is not a mathematical negative answer.

The default command-line worker has a wall-clock limit. The general field construction also has degree and enumeration limits. A timeout never becomes **NotSolvable**. The theoretical completeness statement concerns the uncapped general algorithm and exact terminating primitives; it is not a guarantee that a default invocation will finish.

2 What the supplied notebook contributes

2.1 Inspection and provenance

The archive `FindExtension.zip` contains `FindExtension.nb`. We statically inspected its notebook expression and reconstructed 220 input cells without executing the notebook. The reconstruction is included as `validation/notebook_inputs_static.txt`. It is an audit aid, not a replacement executable notebook: front-end box constructs are not all reconstructed as ordinary Wolfram source.

The original archive SHA-256 is

381e80382e8736aa00e18d8fd48c6f391b9ba0b15e80ff7621e7589147174930.

The originating question supplies the sextic and quintic examples treated in Section 4 [1]. The notebook contains considerably more experiments, including higher-degree examples. We do not claim to have solved every expression or reproduced every notebook computation.

2.2 Extension discovery as a resolvent construction

The functions `findExtension` and `findExtension3` seek coefficients of quadratic or cubic factors over an extension. For example, dividing $f(X)$ by $X^2 + pX + q$ gives a remainder

$$A(p, q)X + B(p, q).$$

A factor requires $A = B = 0$. Eliminating q yields a condition on p . If the chosen quadratic has roots α_i, α_j , then $p = -(\alpha_i + \alpha_j)$. Thus the elimination explores a pair-sum resolvent. This is a principled and useful route to intermediate fields.

There is nevertheless no reason for an arbitrary resolvent root to have low degree, generate a radical extension, or lead to a favorable factorization. A generic pair-sum resolvent can have degree $\binom{n}{2}$. Different roots can correspond to different subgroup orbits. Choosing the shortest-looking coefficient is a useful heuristic, not a completeness argument. Cubic-factor coefficients have the analogous issue.

2.3 Specific limitations visible in the input cells

The first nine input cells expose the core experimental workflow. **project** looks for lower-degree real or imaginary parts of conjugates; such projections can help but cannot characterize all solvable extensions. **factor** and **solve** select candidates using **PossibleZeroQ** and then take a first element. An empty result is not handled uniformly, and a plausible zero test is weaker than an explicit exact equality certificate.

The **roots** helper uses a minimal-polynomial degree while retaining the original **Root** defining function. Those two data sources need not describe the same indexing problem for a reducible or nonminimal defining polynomial. The package instead canonicalizes the selected algebraic number before enumeration.

The **fuzz** routine samples sums of conjugates, calls **RootApproximant** on a 400-digit approximation, and uses a heuristic zero check. Its unbounded loop is suitable for exploration, but a fixed numerical precision cannot establish an algebraic identity for unrestricted degrees and heights. Nor does failing to discover a low-degree sum prove nonsolvability.

The simplification routine uses random transformations, increasing time budgets, large custom complexity penalties, and an explicitly privileged degree-nine **Root**. These choices optimize particular experiments rather than define a general mathematical output criterion. The literal symbol **Simplify**'**SimplifyCount** in the extension routines also differs from **Simplify**'**SimplifyCount** elsewhere. Undocumented implementation symbols and inconsistent spellings should not be a package interface.

The replacement does not discard the notebook's insight. It separates *discovery*, *exact algebraic validation*, *embedding selection*, and *presentation*. Compact structural methods are tried first; an unrelated complete construction supplies a principled fallback.

3 Compact structural constructions

3.1 Translation, binomials, and monomial substitution

For a monic polynomial of degree n with coefficient a_{n-1} , put $s = a_{n-1}/n$ and $Y = X + s$. The translated polynomial $g(Y) = f(Y - s)$ has zero coefficient of Y^{n-1} . This exposes many hidden patterns.

If $g(Y) = Y^n + b$, all roots are

$$X_k = \zeta_n^k (-b)^{1/n} - s, \quad \zeta_n = (-1)^{2/n}, \quad 0 \leq k < n.$$

If the positive exponents of g have greatest common divisor $d > 1$, then $g(Y) = h(Y^d)$. Solve $h(T) = 0$ first, and for each root t use all values $Y = \zeta_d^k t^{1/d}$. Irreducibility of the original polynomial excludes unwanted zero roots and repeated factors in the cases passed to the compact solver. The implementation verifies the final candidates independently rather than relying only on pattern recognition.

3.2 Dickson polynomials and coherent radical branches

Define

$$D_0(X, c) = 2, \quad D_1(X, c) = X, \quad D_n(X, c) = XD_{n-1}(X, c) - cD_{n-2}(X, c).$$

An induction gives the identity

$$D_n(u + c/u, c) = u^n + (c/u)^n. \quad (1)$$

Indeed, multiplying the identity at $n - 1$ by $u + c/u$ and subtracting c times the identity at $n - 2$ cancels the two cross terms.

Suppose the translated polynomial is $D_n(Y, c) + b$, where $c \neq 0$. Choose

$$U = \frac{-b + \sqrt{b^2 - 4c^n}}{2}, \quad u = U^{1/n}.$$

Use the other sign if necessary to avoid $U = 0$. Since $U^2 + bU + c^n = 0$, the values

$$Y_k = \zeta_n^k u + \frac{c}{\zeta_n^k u}, \quad 0 \leq k < n, \quad (2)$$

are roots. When $b^2 \neq 4c^n$, they are distinct: equality for two distinct values v, w with $v^n = w^n = U$ forces $vw = c$, and hence $U^2 = c^n$, a contradiction.

The denominator in (2) is important. Taking two independent principal n th roots need not preserve the required product $uv = c$. Setting the second summand to c/u enforces that relation exactly. The pattern is detected by coefficient comparison: the Y^{n-2} coefficient is $-nc$, and the remaining coefficients are checked against the recurrence.

3.3 Generalized reciprocal reduction

For a monic polynomial of degree $2m$, seek

$$f(X) = X^m h(X + c/X), \quad c \in \mathbb{Q}^*. \quad (3)$$

Necessarily $f(0) = c^m$. There are at most two rational candidates for c obtained from this equation. For each, a triangular coefficient comparison determines a candidate h : subtract the appropriate multiples of $X^m(X + c/X)^j$ for $j = m, m - 1, \dots, 0$. Accept only an exact zero remainder.

For a root y of h , the original roots are

$$X = \frac{y \pm \sqrt{y^2 - 4c}}{2}.$$

This strictly lowers the polynomial degree before lifting through quadratics. The cases $c = 1$ and $c = -1$ include the familiar reciprocal and anti-reciprocal symmetries, but the implementation tests the more general rational identity.

3.4 Cyclotomic and low-degree cases

A primitive N th root of unity is already an arithmetic radical, $(-1)^{2a/N}$ with $\gcd(a, N) = 1$. For a recognized cyclotomic polynomial of degree d , the conductor can be found in a finite range $N \leq 2d^2$. To justify the bound, observe that

$$\frac{\varphi(p^e)^2}{p^e} = p^{e-2}(p-1)^2 \geq 1$$

for odd primes, while the factor for 2^e is at least $1/2$. Multiplication over prime powers gives $\varphi(N)^2 \geq N/2$.

For remaining degrees at most four, the compact backend uses SymPy’s explicit polynomial-root formulas and rejects any expression outside the arithmetic-radical grammar. These formulas are candidate generators; they do not bypass final verification.

3.5 Structural certificates and a dependency edge case

For each candidate e , the implementation computes an exact annihilating polynomial P_e and checks its relation to the irreducible target f . It then identifies e among the exact roots of f using `Poly.same_root`. SymPy’s implementation uses a lower bound for root separation together with bounded-error evaluation, not a chosen decimal tolerance [8]. Both arguments must already be roots of the polynomial supplied to that test.

A regression uncovered a useful reason not to assume that every backend call named `minpoly` returns an irreducible result: in the tested SymPy 1.14.0 environment,

$$\text{minpoly}(-(-1)^{1/11})$$

returns the reducible annihilator $X^{11} - 1$, whereas the true minimal polynomial is Φ_{11} . Accordingly, the code `squarefree-reduces` the returned polynomial, verifies $f \mid P_e$, and uses P_e —not f before membership has been established—for the separation test against the known roots of f . A unique match proves both membership and the conjugate. The certificate records the computed annihilator as well as the established target minimal polynomial.

This structural verifier trusts SymPy’s exact annihilator and bounded-error root-comparison primitives. The general verifier in Section 9 instead implements its own rational-rectangle embedding and branch checks. These are distinct validation paths, and neither is advertised as a formally verified proof-assistant kernel.

4 The original examples, and a notebook octic

4.1 The sextic reduces to a cubic and a quadratic

Let

$$f(X) = X^6 + X^4 - X^3 - X^2 - 1.$$

A direct identity is

$$f(X) = X^3((X - X^{-1})^3 + 4(X - X^{-1}) - 1). \quad (4)$$

Thus $y = X - X^{-1}$ satisfies $y^3 + 4y - 1 = 0$. Put

$$u = \left(\frac{9 + \sqrt{849}}{18} \right)^{1/3}, \quad y = u - \frac{4}{3u}, \quad \boxed{\alpha = \frac{y + \sqrt{y^2 + 4}}{2}}. \quad (5)$$

All the radicals displayed here are positive real radicals. The equation for u^3 gives $y^3 + 4y = 1$ by (1). Because $3y^2 + 4 > 0$, the cubic has exactly one real root; it is positive since its values at 0 and 1 have opposite signs. The two corresponding real roots of f have opposite signs and product -1 . Consequently the positive value in (5) is the second real root, namely the original Wolfram input


```
Root[-1 - #^2 - #^3 + #^4 + #^6 &, 2]
```

Its value is approximately 1.13068544546204. This approximation is a diagnostic, not the proof. The structural implementation constructs and certifies all six conjugates, not only this real branch.

4.2 The quintic is a Dickson equation

After division by 5, the second polynomial is

$$X^5 - 5X^3 + 5X + \frac{6}{5} = D_5(X, 1) + \frac{6}{5}.$$

Let

$$u = \left(\frac{-3 + 4i}{5} \right)^{1/5}, \quad \boxed{\beta = u + u^{-1}}. \quad (6)$$

Then $u^5 + u^{-5} = -6/5$ and (1) proves the quintic equation. Since $|(-3 + 4i)/5| = 1$, the two terms in (6) are conjugate; this agrees with the paired-fifth-root form in the originating question. The reciprocal version makes their compatibility explicit.

Writing $(-3 + 4i)/5 = e^{i\theta}$, with $\pi/2 < \theta < \pi$, the five values are $2\cos((\theta + 2\pi k)/5)$. This trigonometric observation is used only to explain their realness and ordering; the output uses radicals. For $k = 0$, the angle lies between $\pi/10$ and $\pi/5$, closer to zero modulo 2π than the other four angles. Hence (6) is the largest root, approximately 1.80705999508542, and matches

```
Root[6 + 25 # - 25 #^3 + 5 #^5 &, 5]
```

All five conjugates were independently matched in the Python tests. Exact small-degree Galois computations give group orders 24 for the sextic and 20 for this quintic, both solvable; these are computational results recorded in the validation material, not conclusions inferred from the numerical values.

4.3 A degree-eight example from the notebook

The notebook includes

$$F(X) = X^8 + 8X^7 + 20X^6 + 8X^5 - 36X^4 \\ - 48X^3 - 15X^2 + 2X + 1.$$

Translation gives

$$F(Y - 1) = Y^8 - 8Y^6 + 14Y^4 - 7Y^2 + 1 = h(Y^2), \quad h(T) = T^4 - 8T^3 + 14T^2 - 7T + 1.$$

The compact solver handles the quartic and lifts by square roots. For example, the smallest root is

$$-1 - \sqrt{2 + \frac{\sqrt{5}}{2} + \sqrt{\frac{15}{4} + \frac{3\sqrt{5}}{2}}}.$$

All eight output expressions were certified. This demonstrates that the notebook was used beyond the two public examples, while not claiming coverage of its entire experimental collection.

5 Exact splitting fields without numerical recognition

5.1 Representation and trusted primitives

An absolute number field is represented as

$$K = \mathbb{Q}[T]/(F(T)), \quad d = \deg F,$$

where F is monic irreducible. Elements are rational coefficient vectors in the basis $1, t, \dots, t^{d-1}$. Multiplication is polynomial multiplication followed by remainder reduction modulo F ; inversion uses exact field arithmetic. The implementation uses SymPy's algebraic-field polynomial domain for these operations and for factorization over K [6].

The construction does not use `RootApproximant`, decimal matching, or an external system's precomputed splitting-field command. It needs exact polynomial factorization over explicitly represented number fields, rational linear algebra, finite permutation-group operations, and exact isolation of roots of rational polynomials. These are the computational trust assumptions behind the theorem.

5.2 Adjoining a root in a quotient field

Suppose $g(B) \in K[B]$ is a monic irreducible factor of degree $m > 1$. Form

$$E = K[B]/(g(B)), \quad D = dm.$$

A rational basis is $t^i b^j$ with $0 \leq i < d$, $0 \leq j < m$. Multiplication is explicit: reduce b^m using g , and reduce coefficient products using F .

Seek a primitive element $\eta = b + ct$ for integers $c = 0, 1, 2, \dots$. Express the vectors

$$1, \eta, \dots, \eta^{D-1}$$

in the tensor basis and place them in the columns of a rational matrix B_c . When B_c is invertible, calculate

$$v = B_c^{-1}[\eta^D], \quad F_{\text{new}}(T) = T^D - \sum_{j=0}^{D-1} v_j T^j.$$

The same inverse gives the expressions of the old t and new b in powers of η . Both defining relations are then checked again after transport to the new absolute field.

Proposition 5.1 (Termination of primitive-element search). *For separable $E/\mathbb{Q} = \mathbb{Q}(t, b)$ of degree D , all but finitely many rational c make $b + ct$ primitive. In particular, at most $\binom{D}{2}$ values of c are excluded by collisions of distinct embeddings.*

Proof. There are D embeddings of E into an algebraic closure. Distinct embeddings cannot agree on both t and b . Equality of their values on $b + ct$ either is impossible for every c , or determines a single c . Avoid the finitely many collision values. Then $b + ct$ has D distinct conjugates and degree D , so its first D powers form a basis. $\square$

This proves that the integer loop is finite, even though the implementation does not need to compute the excluded values. Irreducibility of g ensures that the quotient is a field; using a reducible factor here would invalidate the argument.

5.3 Building the splitting field

Begin with $K = \mathbb{Q}(\alpha_1)$ for an arbitrary root of the irreducible input f . Factor f over K . If a nonlinear irreducible factor remains, adjoin one of its roots by the preceding construction and repeat. Each extension adds a root not already in the field. When every factor is linear, the resulting field is generated by roots of f and contains all of them, so it is a splitting field L .

At most $n - 1$ further root adjunctions are necessary after the first root; $[L : \mathbb{Q}] \leq n!$. The useful practical bound is often much smaller, but even $n!$ makes the theoretical finiteness clear. Selecting the least-degree remaining factor is a size heuristic; it is not needed for correctness.

A primitive generator is tracked as an integer linear combination of selected original roots. Under the replacement $\eta = b + ct$, the old weights are multiplied by c and the new root receives weight 1. This simple bookkeeping later avoids factoring an enormous primitive-element polynomial merely to find its automorphisms.

6 A cyclotomic base that preserves the solvability decision

Let L be the splitting field just constructed and let $d_L = [L : \mathbb{Q}]$. Define

$$N = \text{rad}(d_L) = \prod_{p|d_L} p, \quad C = \mathbb{Q}(\zeta_N), \quad M = L(\zeta_N).$$

Here ζ_N is any primitive N th root of unity. Factor Φ_N over L . A linear factor supplies ζ_N immediately; otherwise adjoin a root of a nonlinear irreducible factor using the same quotient-field construction. The resulting field M is the compositum of two Galois extensions of $\mathbb{Q}$.

Proposition 6.1. *Write $G = \text{Gal}(L/\mathbb{Q})$ and $H = \text{Gal}(M/C)$. Then H is solvable if and only if G is solvable. Every prime divisor of $|H|$ divides N , and C contains the corresponding primitive prime-order roots of unity.*

Proof. Restriction identifies H with $\text{Gal}(L/(L \cap C))$. The intersection $L \cap C$ is Galois and abelian over $\mathbb{Q}$: it is a subextension of the abelian extension $C/\mathbb{Q}$. Thus this subgroup is normal in G and its quotient is abelian. A subgroup of a solvable group is solvable, and an extension of a solvable group by an abelian group is solvable. Finally, $|H|$ divides $|G| = d_L$, proving the assertion about prime divisors. $\square$

This choice avoids adjoining an unnecessarily huge collection of roots of unity. More importantly, it avoids a circular specification such as “first construct radicals for the full cyclotomic compositum by a method requiring still more roots of unity.” We start the radical descent *over* C , whose generator is already the explicit rational power $(-1)^{2a/N}$ for a certified exponent a . Prime factors of $[C : \mathbb{Q}] = \varphi(N)$ do not need to be added to N : they do not occur as required steps of the relative descent unless they already divide $|H|$.

The absolute degree satisfies

$$D = [M : \mathbb{Q}] \leq d_L \varphi(N), \quad |H| = D/\varphi(N).$$

The first bound can still be large. It is a finiteness argument, not a performance estimate.

7 Computing the Galois action explicitly

7.1 Images of a tracked primitive generator

After the preceding constructions, the primitive generator has an exact description

$$\theta = \sum_{j=1}^s w_j \alpha_{i_j} + c \zeta_N, \quad w_j, c \in \mathbb{Z}, \quad (7)$$

where the α_i are distinct roots of f . Some weights may be zero. An automorphism fixing ζ_N sends θ to a value obtained by replacing the selected roots in (7) with distinct roots of f .

Enumerate these finitely many candidates η . For each, check in the exact field M that its primitive defining polynomial vanishes, that substitution $\theta \mapsto \eta$ fixes ζ_N , and that it induces the recorded permutation of all original roots. A root of the irreducible primitive polynomial defines a $\mathbb{Q}$ -embedding of M into itself, hence an automorphism. Conversely, every automorphism in H appears among the candidates.

The action on the original roots is faithful: an element fixing all roots and ζ_N fixes θ by (7). Thus the accepted permutations are exactly H as a subgroup of S_n . The code checks both the number of accepted images and the permutation-group order against $D/\varphi(N)$. It may stop once this known number has been found. The candidate bound is a resource guard, not a test of solvability.

7.2 Small-degree and modular shortcuts

SymPy's direct `galois_group` routine is documented for irreducible polynomials of degree at most six [6]. The package uses it as an optional early obstruction test. It does not rely on that degree-limited routine for its general construction. If the routine exhausts its bounded transformation attempts, the construction continues rather than interpreting this as a mathematical failure.

For irreducible prime degree $n \geq 5$, another exact shortcut is useful. At a prime of squarefree reduction, the degrees of the irreducible factors modulo that prime give a cycle type in the Galois group. If there is precisely one degree-two factor and every other degree is odd, an odd power of that permutation is a transposition. A transitive group of prime degree is primitive. A primitive subgroup containing a transposition is all of S_n , so it is nonsolvable.

For completeness of the last group-theoretic step, form the graph whose edges are the transpositions conjugate to the given one. Its connected components form a block system. Primitivity makes this graph connected. Transpositions along the edges of a connected graph generate S_n .

The example $X^7 - X - 1$ is irreducible modulo 2. Modulo 3 it has the factorization

$$(X^2 + X - 1)(X^5 - X^4 - X^3 - X + 1),$$

with irreducible factors of degrees 2 and 5. The stored certificate checks this factorization, irreducibility, and the hypotheses above. The search over small reduction primes is only a shortcut; finding no such prime has no negative interpretation.

7.3 A prime-index normal series

Once the actual finite group H is available, compute its derived series to decide solvability. For a solvable group, refine to a composition series

$$H = H_0 \triangleright H_1 \triangleright \cdots \triangleright H_r = 1, \quad H_{i-1}/H_i \cong C_{p_i} \quad (8)$$

with prime p_i . SymPy supplies finite permutation-group operations and a composition-series implementation [7]. The package explicitly checks normality in the *immediately preceding subgroup* and primality of each index. It does not assume that every H_i is normal in the original H .

Put $E_i = M^{H_i}$. Then

$$C = E_0 \subset E_1 \subset \cdots \subset E_r = M, \quad [E_i : E_{i-1}] = p_i,$$

and each adjacent extension is cyclic. These are the extensions that will become individual radicals.

8 Kummer generators by rational linear algebra

8.1 The linear-algebra lemma

Fix one step $J \triangleright K$ of prime index p , and put $F_0 = M^J$, $F_1 = M^K$. Choose $\sigma \in J \setminus K$ and a primitive p th root $\lambda \in C$. The induced action of σ generates $\text{Gal}(F_1/F_0)$.

Lemma 8.1 (A nonzero eigenspace supplies the radical). *There exists $0 \neq b \in F_1$ with $\sigma(b) = \lambda b$. For every such b ,*

$$b^p \in F_0, \quad F_1 = F_0(b).$$

The set of solutions of this eigenvalue equation in F_1 is one-dimensional over F_0 .

Proof. Distinct automorphisms of a field are linearly independent as maps over that field. Consequently the operator

$$P_\lambda = \sum_{j=0}^{p-1} \lambda^{-j} \sigma^j$$

is not the zero map on F_1 . Its image consists of λ -eigenvectors; choose a nonzero one. The identity $\lambda^p = 1$ makes b^p fixed by σ , hence in F_0 . Since $\lambda \neq 1$, $b \notin F_0$; the extension has prime degree, so b generates it. The ratio of any two nonzero eigenvectors is fixed by σ , proving the dimension statement. $\square$

This is the constructive content of cyclic/Kummer extension theory [2]. The implementation does not need to guess an element for which a resolvent is nonzero: it computes the entire solution space by a nullspace calculation.

8.2 Why the matrices have rational, not algebraic, entries

Use the absolute basis $1, \theta, \dots, \theta^{D-1}$ of M . For an automorphism g , let S_g be its rational matrix; column j is the vector of $g(\theta)^j$. Let T_λ be the rational matrix of multiplication by λ .

The desired vector $v = [b]$ solves

$$(S_h - I)v = 0 \quad (h \text{ among generators of } K), \quad (S_\sigma - T_\lambda)v = 0. \quad (9)$$

The multiplication matrix T_λ is essential. Replacing it by an algebraic scalar in a rational-coordinate equation without extending the coefficient domain would mix two different linear spaces. Equation (9) stays entirely over $\mathbb{Q}$.

By Lemma 8.1, its rational nullspace has dimension $[F_0 : \mathbb{Q}] = D/|J|$. The code verifies this dimension, chooses a nonzero vector, and clears rational denominators and common integer content. It then rechecks the eigenvalue and invariance identities exactly.

8.3 Converting the power into the preceding radical field

Maintain a rational basis B_0 of F_0 together with its already constructed radical expressions. For the first step,

$$B_0 = (1, \zeta_N, \dots, \zeta_N^{\varphi(N)-1}).$$

Since $\delta = b^p \in F_0$, solve for its coordinates in B_0 using a reusable exact left inverse. An exact residual check proves that it lies in the claimed span; merely obtaining a least-squares solution would not suffice.

The next basis is

$$B_1 = \{ab^j : a \in B_0, 0 \leq j < p\}.$$

It is independent and spans F_1 because $1, b, \dots, b^{p-1}$ is an F_0 -basis. Append one radical definition for b , with its principal branch fixed as in the next section, and update the expressions in the same order as the field basis. Repeat until a basis of all of M has been expressed in radicals. Finally express each original root in this basis.

This procedure avoids solving a general high-degree equation in an unspecified intermediate field. Every nonlinear operation introduced at a descent step is exactly one prime-order radical of an element already expressed in previous radicals.

9 Exact embeddings and principal-power branches

9.1 Abstract fields may change their complex embedding

During adjunction we freely replace a field by an isomorphic absolute presentation and choose an arbitrary isolated root of its defining polynomial, specifically index zero. The abstract maps are checked, but that chosen complex embedding need not extend the embedding selected in the preceding presentation.

This is harmless because the final field contains *all* conjugates of the original input. Only after the field construction do we identify their actual embedded values and match them to the input root indices. Trying to preserve an unverified numerical root label at every primitive-element change would be considerably more error-prone.

9.2 Rational rectangles and exact root matching

A complex enclosure is a rectangle

$$[a_-, a_+] + i[b_-, b_+], \quad a_{\pm}, b_{\pm} \in \mathbb{Q}.$$

Addition and multiplication use rational interval arithmetic. A rational polynomial in the primitive element is enclosed by Horner evaluation on a refined isolating rectangle for θ . These enclosures converge to the embedded value as the primitive rectangle shrinks, even when their initial overestimation is large.

Before matching a field element a to the isolated roots of f , the verifier checks $f(a) = 0$ by exact field arithmetic. It then refines the enclosure of a and every isolated root until the former intersects exactly one of the latter. The match is certain: the true value belongs to the isolating rectangle of its true root, so that rectangle must intersect its enclosure. Distinct roots have positive separation, guaranteeing eventual uniqueness.

The implementation uses SymPy's rational root-isolation objects through a small adapter, `RootBox`. Its private calls `CRootOf._get_interval()` and `refine_size()` are deliberately localized and the dependency is pinned to SymPy 1.14.0. An incompatible API is a software error, not evidence about radical solvability.

9.3 Complex conjugation and sign decisions

Because $M/\mathbb{Q}$ is Galois, complex conjugation acts on M . Formula (7) gives its image on θ : conjugate each selected original root, and replace ζ_N by its inverse. The matches just established identify the conjugate roots exactly. The resulting substitution is checked against the primitive defining polynomial and checked to square to the identity.

For $a \in M$, we can therefore test

$$\Re(a) = 0 \iff a + \bar{a} = 0, \quad \Im(a) = 0 \iff a - \bar{a} = 0$$

by exact field equality. For a nonzero component, refine rational rectangles until its interval is strictly positive or strictly negative. The initial exact zero test is crucial: interval refinement alone might run forever on a branch boundary.

9.4 Fixing the cyclotomic generator

The selected primitive ζ_N has an embedding

$$\zeta_N = e^{2\pi ia/N} = (-1)^{2a/N}, \quad \gcd(a, N) = 1.$$

Match it to a root of Φ_N by the preceding exact method. Its exponent is recovered from the standard root ordering. For nonreal roots, the real part is ordered by $-\min(a, N - a)$, with the negative-imaginary conjugate first at a tie. The case $N = 2$ is checked directly. No value of a transcendental trigonometric function enters the certificate.

9.5 A branch condition expressible by exact signs

Suppose $b^p = \delta$ and $\lambda = \zeta_N^{N/p}$. Exactly one $k \in \{0, \dots, p-1\}$ makes

$$\gamma = b/\lambda^k$$

lie in the principal-root sector $-\pi/p < \text{Arg } \gamma \leq \pi/p$. Then

$$b = \lambda^k \text{pr}_p(\delta). \tag{10}$$

For $p = 2$, the sector criterion is

$$\Re \gamma > 0 \quad \text{or} \quad (\Re \gamma = 0 \text{ and } \Im \gamma > 0).$$

For odd p , let a^{-1} denote the inverse of the certified cyclotomic exponent modulo p , put

$$\omega = \lambda^{a^{-1}} = e^{2\pi i/p}, \quad u = -\omega^{(p+1)/2} = e^{i\pi/p}.$$

The required exact tests are

$$\Re \gamma > 0, \quad \Im(\gamma/u) \leq 0, \quad \Im(\gamma u) > 0. \tag{11}$$

They describe the wedge between its two boundary rays. The upper boundary is included and the lower one excluded, precisely matching the chosen principal argument. The nonzero eigenvector hypothesis excludes the vertex. All rotations used for odd p belong to M ; the $p = 2$ case avoids needing to adjoin i just for a sector test.

The verifier recomputes these signs. It does not trust a numerical argument, a recorded decimal value, or the branch number supplied by the solver.

10 The complete algorithm and its correctness

10.1 Pseudocode

```

RadicalsForIrreducible(f):
  L, roots := exact splitting field of f over Q
  N := product of distinct prime divisors of [L:Q]
  M := L(zeta_N)
  H := exact automorphisms of M fixing zeta_N
  if H is not solvable:
    return NotSolvable with an exact obstruction
  choose a prime-index normal series H = H0 > ... > Hr = 1
  certify the embedding of M and of zeta_N
  B := [1, zeta_N, ..., zeta_N^(phi(N)-1)]
  for J > K in the series, with index p:
    solve the rational invariance/eigenvector system for beta != 0
    delta := coordinates of beta^p in B
    certify beta = zeta_N^k * PrincipalRoot(delta, p)
    append this radical definition
    B := [b * beta^j for j in 0..p-1 for b in B]
  express every original root in B
  return the radical program, root indices, and certificate

```

Theorem 10.1 (Uncapped constructive completeness). *Assume exact terminating algorithms for factorization over the explicitly constructed number fields, rational linear algebra, rational-polynomial root isolation, and finite permutation-group operations. With degree, enumeration, and time caps removed, the general algorithm terminates on every irreducible $f \in \mathbb{Q}[X]$. It returns a radical expression for every root exactly when $\text{Gal}(f/\mathbb{Q})$ is solvable, and otherwise returns an exact nonsolvability obstruction.*

Proof. The splitting-field construction makes finitely many root adjunctions. Every primitive-element search terminates by the collision argument. Cyclotomic adjunction is another finite field construction. The possible primitive-generator images form a finite set, exhaust the relative automorphism group, and are tested exactly.

Proposition 6.1 shows that the relative group’s solvability gives the correct answer for f . For a solvable group, the prime-index series is finite. Lemma 8.1 proves existence of a nonzero vector at each step, proves that its prime power belongs to the preceding field, and proves that the enlarged basis is correct. The radical assignment has the required embedding by (10) and the exact sector conditions. Root matching and nonzero sign decisions terminate by root separation and shrinking rational enclosures; exact boundary-zero tests prevent an inconclusive refinement loop. Induction expresses all of M in the radical program, hence every root of f .

For a nonsolvable relative group, its faithful action on the splitting roots embeds it in the original splitting group. Thus the obstruction is genuine, and Theorem 1.2 excludes radicals. $\square$

Remark 10.2 (What this theorem does not say). It supplies a complete *algorithm*, not a polynomial-time bound in the input degree, not a promise that SymPy has no bugs, and not an assertion that all released code paths were exhaustively tested. In particular, the default capped program may return **Unknown** for a solvable polynomial. The automatic structural front end is an optimization; requesting `method='general'` selects the algorithm described by the theorem directly.

11 Certificates and independent checks

11.1 The radical program

The JSON grammar is deliberately small:

```
[ "q", numerator, denominator ]
[ "ref", "previous_name" ]
[ "add", expression, ... ]
[ "mul", expression, ... ]
[ "pow", expression, numerator, denominator ]
```

Denominators are positive integers. A reference must name an earlier definition. Neither executable Python strings nor arbitrary Wolfram expressions are accepted. The Wolfram decoder constructs only the corresponding arithmetic expressions; it does not call `ToExpression` on backend output.

For a general construction, the first definition is the certified cyclotomic generator. Each subsequent definition introduces the rotated principal prime root of an expression in previous definitions. Each final root is a rational-coordinate polynomial in these named values. The field polynomial is audit data, not a hidden operation needed to evaluate the output program.

11.2 Positive general certificates

The certificate stores the irreducible defining polynomial of M , its chosen root index, the original roots as coefficient vectors, the cyclotomic element and exponent, the tracked primitive-generator description, and the Kummer generators. Each stage records the prime, radicand, branch number, and sector-sign witnesses.

The separate `verify_constructive` routine does not recompute the Galois group. It checks irreducibility, the complete distinct root set, the primitive description, the root-of-unity order and embedding, every power identity, every principal-sector decision, and every output coordinate. This smaller task proves a *positive* answer even if the discovery procedure used an inefficient series or an unnecessarily large field. A successful certificate cannot smuggle in a nonsolvable algebraic constant: its only initial constant beyond $\mathbb{Q}$ is an explicitly certified root of unity, and every later field value is justified by an actual arithmetic radical assignment.

The test suite changes branch numbers, output expressions, and conjugate indices and requires rejection. Reusing exact-arithmetic libraries in both construction and verification is part of the trust boundary; the verifier is independent of the discovery decisions, not independent of all software dependencies.

11.3 Negative certificates

The modular prime-degree certificate is particularly small: it records a reduction prime and the irreducible modular factors. Verification checks the input's irreducibility and the cycle-pattern argument.

For a small-degree direct Galois result, the verifier recomputes that exact group. For the general negative path, the certificate contains field generators, splitting roots, the cyclotomic element, and images and permutations of group generators. Exact substitution proves that these are automorphisms fixing the cyclotomic base. The primitive description proves that no unrelated generators were introduced. A nonsolvable subgroup in their faithful action on the original roots already suffices; a negative verifier need not prove that every possible automorphism was enumerated.

The general negative-certificate path is implemented, but the executed negative regressions use the small-degree and modular shortcuts. No claim is made that a factorial-size nonsolvable splitting field was successfully constructed during testing.

12 Package interfaces and reproducible use

12.1 Installation and Python API

The primary executable implementation is Python with SymPy 1.14.0. It was tested on Python 3.13. Install the pinned dependency in a virtual environment if desired:

```
python -m pip install -r requirements.txt
python -m unittest discover -s tests -v
python examples/demo.py
```

From a Python program, add the delivered `python` directory to the import path and use:

```
import sympy as s
from api import root_radicals, all_radicals, verify, selected_expression
x = s.Symbol('x')
result = root_radicals(x**6 + x**4 - x**3 - x**2 - 1, 1)
assert result['status'] == 'Success'
assert verify(result)
answer = selected_expression(result)
```

Python indices are zero-based and use SymPy's `CRootOf` conventions. Thus the positive sextic root has index 1, while the largest quintic root has index 4. For a reducible polynomial, the result contains the conjugates of the *selected factor*, not necessarily every root of the original polynomial.

`all_radicals` requires an irreducible rational polynomial. Its `method` argument accepts `'auto'`, `'general'`, or `'structural'`. Structural-only search never treats a missing pattern as nonsolvability. `root_radicals` accepts reducible and repeated-root polynomials and performs exact selected-factor reduction. Machine-precision coefficients and symbolic parameters are rejected rather than silently rationalized.

To remove mathematical search caps from the general API:

```
result = all_radicals(f, method='general',
```

```
max_field_degree=None, max_candidates=None)
```

This invocation has no library-imposed wall-clock bound. It can consume very substantial time and memory.

12.2 Command-line interface and saved certificates

The CLI accepts high-to-low coefficient arrays. Rational coefficients must be integer values or exact fraction strings. For example:

```
python python/cli.py --coefficients "[1,0,1,-1,-1,0,-1]" \
  --index 1 --output sextic.json
python python/cli.py --coefficients "[5,0,-25,0,25,6]" \
  --index 4 --output quintic.json
python python/cli.py --verify-file sextic.json
```

Line continuation syntax depends on the user's shell; the commands can be entered on one line. The default CLI independently verifies both positive and negative results. Its `-no-verify` switch skips only that additional recheck, not the construction's exact algebraic and branch checks.

The default wall-clock limit is 120 seconds, general field-degree cap 128, and candidate cap 1,000,000. The timeout includes construction and the optional recheck in a killable worker. Use `-timeout 0`, `-max-field-degree 0`, and `-max-candidates 0` to disable these limits. A hard memory limit is not provided. A `-json` mode reads a single structured request from standard input; progress goes to standard error, preserving machine-readable standard output.

12.3 Wolfram Language adapter

Load the package and call it as follows:

```
Get["/path/to/RadicalRoot/wolfram/RadicalRoot.wl"];
r = Root[-1 - #^2 - #^3 + #^4 + #^6 &, 2];
a = RadicalRoot[r,
  "UseNative" -> False,
  "PythonExecutable" -> "/path/to/python"];
RootReduce[a - r]
```

The expected final result is zero. On Windows, the executable can be an absolute `python.exe` path. The native `ToRadicals` fast path is enabled by default; disabling it forces use of the supplied backend.

The adapter obtains the minimal polynomial, passes rational coefficients through a shell-free `RunProcess` argument list, requests backend verification, and decodes the whitelist expression tree. It does *not* assume that Python and Wolfram order nonreal conjugates identically. Instead it tests candidate expressions against the requested number using `RootReduce[expression-input] === 0`. Wolfram documents canonical reduction and exact algebraic equality uses of this function [4]; the external-process call follows the documented structured-argument interface [5].

Use `"Method" -> "General"` to force the general constructor, or `"Return" -> "Data"` to retain the backend proof data. The options `"TimeLimit"` and `"VerificationTimeLimit"` control the two

time budgets. General search caps are set with "MaximumFieldDegree" and "MaximumCandidates". Infinity removes the corresponding backend search cap. The backend time limit is not a single deadline for all Wolfram-side normalization and final candidate comparisons; those have separate controls. A Wolfram failure to finish its final equality check is reported as an uncertified embedding, not as a negative solvability result.

Testing limitation. The connected Wolfram evaluation service returned an HTTP 404 error during attempts to use it. No native Wolfram kernel was available locally. Consequently the adapter and its supplied `.wlt` tests were source-reviewed but not executed. The equivalent Python implementation is the tested deliverable; this distinction should be retained when distributing the package.

13 Executed validation and observed limits

13.1 What was actually run

The released regression suite contains 20 named tests, with multiple polynomial subcases in several tests. The final recorded run passed all 20 in about 41 seconds on the delivery environment. This is a reproducibility observation, not a cross-machine performance benchmark. The suite tests exact positive certificates, negative certificates, selected factors of reducible polynomials, repeated roots, branch-boundary cases, invalid inputs, the JSON protocol, hard timeouts, resource caps, and tampered certificates.

The general constructor was separately run on the following cases. All rows marked verified have a completed construction and an accepted positive certificate; the quintic certificate files and run logs are included.

Polynomial	$\deg f$	$[L : \mathbb{Q}]$	$[M : \mathbb{Q}]$	$ H $	Result
$X^2 - 2$	2	2	2	2	Verified
$X^2 + 1$	2	2	2	2	Verified
$X^3 - 2$	3	6	6	3	Verified
$X^3 - X - 1$	3	6	12	6	Verified
$X^3 - 3X + 1$	3	3	6	3	Verified
$X^4 - 2$	4	8	8	8	Verified
$X^4 - 10X^2 + 1$	4	4	4	4	Verified
$X^5 - 2$	5	20	20	5	Verified
$X^5 + X^4 - 4X^3 - 3X^2 + 3X + 1$	5	5	20	5	Verified

The cyclic quintic is important: it exercises a genuine prime-five Kummer step in an explicitly constructed degree-twenty field, rather than delegating to a quartic formula or simply recognizing a binomial. The pure quintic was also deliberately forced through the general construction even though the compact solver can solve it immediately.

The compact path completed both original examples, every conjugate of the notebook octic in Section 4, shifted binomials, and the degree-ten cyclotomic polynomial Φ_{11} . The negative tests include $X^5 - X - 1$ using its exact Galois group and $X^7 - X - 1$ using the modular transposition certificate. A test of $(X - 2)(X^5 - X - 1)$ confirms that the rational root succeeds while a root of the nonsolvable factor receives a negative result.

13.2 Expensive runs that must not be counted as successes

We also forced the general constructor on the two original examples without using their compact reductions. Each run was externally limited to 480 seconds. Neither produced a completed radical certificate within that limit. The sextic reached an absolute field of degree 48 and was still in automorphism computation; the quintic reached an absolute field of degree 40 and had entered embedding certification. The partial logs are preserved and explicitly labeled incomplete exploratory runs.

These observations are consistent with, but do not weaken, the finite-algorithm theorem. They do show why a useful package needs both structural methods and honest resource reporting. The phrase “general solver” does not imply that every sextic will be cheaper than every quintic, or that all solvable degree-eight inputs resemble the simple notebook octic.

13.3 Limits of the evidence

Finite regression tests cannot establish absence of implementation bugs. The arbitrary-degree theorem is proved for the algorithm, while the executed examples exercise only modest absolute field degrees. The tested general positive fields reached degree twenty; larger completed positive general constructions are not claimed here. The Wolfram adapter has no native execution evidence. The private SymPy root-isolation interface is version-sensitive. The Python source and proof checkers are delivered so that these limitations can be investigated rather than hidden behind a success label.

14 Performance, expression quality, and further development

14.1 Where the reference implementation spends time

The dominant costs need not be radical extraction. Repeated factorization in large absolute fields, dense rational matrix inversion, coefficient swell in primitive-element substitutions, automorphism-image evaluation, and interval refinement of high-height primitive presentations can dominate long before the first radical stage is emitted.

The implementation uses dense exact arithmetic and selects the first usable primitive element and a simple nullspace vector. These choices make the code auditable, but they do not minimize discriminants, coefficient heights, or the conditioning of a complex embedding. A large expression in the primitive generator can require very narrow root intervals even when the algebraic number itself is simple.

The radical program is stored as a directed acyclic graph of definitions. Fully substituting all definitions can duplicate large subexpressions many times. `selected_expression` is convenient for small examples, but the JSON program should remain the primary representation for large answers. Arithmetic radicals can also be numerically ill-conditioned, particularly near branch cuts or when large terms cancel; numerical evaluation should not replace the exact certificate.

14.2 Improvements that preserve the mathematical contract

Promising engineering directions are smaller primitive generators obtained by lattice reduction, relative number-field representations that postpone conversion to one absolute field, modular and fraction-free linear algebra, automorphism enumeration along the original adjunction tower, and

more structural transformations before a full splitting field is constructed. A production backend could replace exact arithmetic primitives with faster implementations while retaining the same certificates and their independent checking interface.

The notebook’s factor-coefficient resolvents can be reintroduced as optional discovery plugins. A proposed extension should be accepted only after exact field membership, degree, factorization, and embedding checks; failure of that plugin should then fall back to the complete path. Another useful optimization is to choose Kummer eigenvectors whose powers have short coordinates in the preceding field. This is an expression-optimization problem on top of the existence theorem, not a prerequisite for solvability.

Algorithms specifically designed around known Galois actions and Lagrange resolvents provide relevant related work; for example, Li describes an arbitrary-degree solvable-polynomial radical algorithm under specified Galois-action and root-approximation inputs [9]. This package makes no novelty claim for the solvable-group-to-radicals principle. Its emphasis is an explicit executable field construction, rational eigenspace formulation, branch-aware certificates, and clear separation of mathematical completeness from practical coverage.

14.3 What remains outside the contract

The package does not determine a shortest radical formula, minimize the maximum radical order, or require all intermediate quantities to be real. It does not solve parameter-dependent algebraic functions with branch loci. It does not offer a fast uniform algorithm for all solvable groups. It does not infer an exact algebraic number from a floating-point input. Those are different problems and should not be silently conflated with conversion of an already exact numerical `Root`.

15 Conclusion

A systematic replacement for experimental extension hunting is possible. The central distinction is between a *complete construction* and a *uniformly efficient implementation*. Exact splitting fields, a cyclotomic base, an explicit solvable group action, and prime-index eigenspaces give a finite route to arithmetic radicals whenever such radicals exist. Exact complex embedding and principal-sector checks turn that existence construction into the requested selected-root expression.

The two original examples additionally show why recognizing structure is worthwhile: one is a cubic followed by a quadratic under $X - X^{-1}$, and the other is a Dickson quintic with a coherent fifth-root formula. The delivered package uses these general patterns, not hardcoded answers to those polynomials, and retains the constructive Galois algorithm as its mathematical fallback. Its negative answers have algebraic reasons; its resource failures are labeled unknown; and its validation record distinguishes executed Python computations from an unexecuted Wolfram adapter.

A File map and reproduction notes

File	Role
<code>python/exact_fields.py</code>	Exact absolute fields, root adjunction, field transport, and coordinates.

File	Role
python/certified_embedding.py	Rational rectangles, conjugation, root indices, and principal sectors.
python/radical_solver.py	General Galois construction and positive/negative proof checkers.
python/structural.py	Compact formulas and annihilator/separation certificates.
python/obstructions.py	Prime-degree modular nonsolvability certificates.
python/radical_ast.py	Whitelisted expression grammar and conversions.
python/api.py	Selected-root and all-conjugate public APIs.
python/cli.py	JSON protocol, rational parsing, worker timeout, saved-result verification.
wolfram/RadicalRoot.wl	Wolfram adapter with native fast path and exact final equality.
tests/test_radicalroot.py	Executed Python regression suite.
tests/RadicalRoot.wlt	Supplied, unexecuted Wolfram tests.
validation/	Selected certificates, logs, notebook reconstruction, and validation notes.
article/RadicalRoot.tex	This source; compile with <code>pdflatex</code> twice.
article/RadicalRoot.pdf	Compiled article.

No third-party computational library is bundled. The original experimental notebook is not modified. Its reconstructed input-cell text is included solely to make the audit traceable. The MIT license accompanying the package applies to the new implementation; it does not relicense the supplied notebook or third-party dependencies.

B The distinction between an expression and its proof

A radical program may have assignments such as

$$z = (-1)^{2a/N}, \quad r_1 = z^{k_1 N/p_1} \delta_1(z)^{1/p_1}, \quad r_2 = z^{k_2 N/p_2} \delta_2(z, r_1)^{1/p_2},$$

followed by a rational-coordinate formula $P(z, r_1, r_2, \dots)$. Every symbol on the right has already been defined. The certificate additionally says that a particular coefficient vector in $\mathbb{Q}[T]/(F)$ represents r_i . This is not a circular definition: the arithmetic program can be evaluated without the field polynomial, while the field polynomial makes exact checking possible.

Conversely, merely returning an expression like “a chosen root of $Y^p - \delta$ ” would be incomplete until the choice were specified. Equation (10) and the sector signs are precisely the information that removes this ambiguity. Retaining the proof data also protects against later simplifications that accidentally replace one branch by another.

References

- [1] Vladimir Reshetnikov, *Why is ToRadicals not able to handle all cases? Is there a workaround?*, Mathematica Stack Exchange, question 34011. <https://mathematica.stackexchange.com/>

- [questions/34011/](#). The supplied `FindExtension.nb` is additional user-provided primary material.
- [2] J. S. Milne, *Fields and Galois Theory*, version 5.10, 2022. In particular, cyclic extensions and Galois's solvability theorem. <https://www.jmilne.org/math/CourseNotes/FT.pdf>.
 - [3] Wolfram Research, *ToRadicals*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/ToRadicals.html>.
 - [4] Wolfram Research, *RootReduce*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/RootReduce.html>.
 - [5] Wolfram Research, *RunProcess*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/RunProcess.html>.
 - [6] SymPy development team, *Number Fields*, SymPy 1.14.0 documentation. <https://docs.sympy.org/latest/modules/polys/numberfields.html>.
 - [7] SymPy development team, *Permutation Groups*, SymPy 1.14.0 documentation. https://docs.sympy.org/latest/modules/combinatorics/perm_groups.html.
 - [8] SymPy development team, *Polynomials Manipulation Module Reference*, especially `Poly.same_root`, and the corresponding implementation in SymPy 1.14.0. <https://docs.sympy.org/latest/modules/polys/reference.html>.
 - [9] Song Li, *An Algorithm for Solving Solvable Polynomial Equations of Arbitrary Degree by Radicals*, arXiv:2203.11602, 2022. <https://arxiv.org/abs/2203.11602>.

Online documentation consulted on 8 September 2026. The code is pinned to the version actually tested; this is not a claim that every future dependency version preserves internal interfaces.